

Objectifs :

- ✓ Maitrise des Threads en Java

Objectif du TP

L'objectif de ce TP est de comprendre les problèmes de concurrence d'accès aux ressources partagées entre plusieurs threads, et d'apprendre à les résoudre en utilisant les mécanismes de synchronisation de Java (synchronized, wait(), notify()).

Contexte

Un distributeur automatique de billets (DAB) est partagé par plusieurs clients. Le DAB contient une certaine quantité d'argent (un solde). Les clients peuvent effectuer deux types d'opérations :

- Retirer de l'argent
- Déposer de l'argent

Lorsqu'un client effectue un retrait, le DAB doit vérifier que le solde est suffisant avant de délivrer les billets. Cette vérification et la délivrance prennent du temps (simulé par un sleep). Le problème est que plusieurs clients peuvent tenter d'utiliser le DAB en même temps, ce qui peut entraîner des incohérences (solde négatif, perte d'argent, etc.).

PARTIE 1 : SANS SYNCHRONISATION (Problème de concurrence)

Travail à faire :

1. Créez une classe **Distributeur** contenant :
 - Un attribut privé fonds de type double initialisé à 5000 DH
 - Un getter et un setter pour fonds
2. Une méthode **retirer(double montant, String nomClient)** qui :
 - Affiche que le client vérifie le solde
 - Vérifie si le solde est suffisant
 - Simule un temps de délivrance des billets avec Thread.sleep(2000)
 - Si le solde est suffisant, retire le montant et affiche le nouveau solde
 - Sinon, affiche un message "Solde insuffisant"
3. Créez une classe **Client** qui implémente **Runnable** contenant :
 - Un attribut **distributeur** de type **Distributeur**
 - Un attribut nom de type String
 - Un attribut montantRetrait de type double
 - Un attribut nombreOperations de type int
 - Un constructeur pour initialiser tous les attributs
 - La méthode run() qui effectue en boucle (nombreOperations fois) :
 - Un appel à la méthode retirer() du distributeur
 - Une pause de 1 seconde entre chaque retrait
4. Créez une classe **Main** qui :
 - Crée un objet Distributeur
 - Crée deux objets Client : Ahmed et Amal
 - Chaque client effectue 3 retraits de 3000 DH
 - Crée deux threads à partir de ces clients
 - Lance les deux threads
 - Exécutez le programme et observez les résultats.

PARTIE 2 : AVEC SYNCHRONISATION

Travail à faire :

1. Créez une classe **DistributeurSecurise** contenant :
 - Un attribut privé fonds de type double initialisé à 5000 DH
 - Une méthode synchronized void **retirer**(double montant, String nomClient) qui :
 - Affiche que le client tente de retirer
 - Tant que le solde est insuffisant :
 - Affiche un message d'attente
 - Utilise wait() pour mettre le thread en attente
 - Simule un temps de délivrance avec Thread.sleep(2000)
 - Retire le montant et affiche le nouveau solde
 - Une méthode synchronized void **deposer**(double montant, String nomClient) qui :
 - Affiche que le client dépose de l'argent
 - Ajoute le montant au solde
 - Affiche le nouveau solde
 - Utilise notifyAll() pour réveiller les threads en attente
 - Une méthode synchronized double **getFonds()** pour consulter le solde
2. Créez une classe **ClientOperation** qui implémente Runnable contenant :
 - Un attribut distributeur de type DistributeurSecurise
 - Un attribut nom de type String
 - Un attribut estDepot de type boolean (true = dépôt, false = retrait)
 - Un attribut montant de type double
 - Un attribut nombreOperations de type int
 - La méthode run() qui effectue en boucle les opérations demandées
3. Créez une classe MainSecurise qui :
 - Crée un objet DistributeurSecurise
 - Crée les threads suivants :
 - Ahmed : effectue 3 retraits de 3000 DH
 - Amal : effectue 2 dépôts de 2000 DH
 - Amal : effectue 1 retrait de 3000 DH
 - Lance tous les threads
 - Crée un thread "afficheur" qui toutes les 3 secondes affiche le solde actuel (thread démon)
4. Exécutez le programme et observez les résultats.